

Decoding “The Tell”: Exploitative Reinforcement Learning and Implicit Opponent Modeling in 5-Card Draw Poker

Alkım Gönenç Efe¹, Sarp Sünbül², Damla Parlakyıldız³

Department of Computer Engineering
Izmir Katip Celebi University
Izmir, Turkey

Abstract—Poker is a canonical benchmark for artificial intelligence research in imperfect-information games, requiring agents to reason under uncertainty, manage hidden state, and adapt strategy to diverse opponents. This paper presents CardShark-RL, a reinforcement learning system for heads-up limit 5-Card Draw Poker that investigates two complementary approaches to opponent adaptation: (A) explicit opponent modeling, in which a one-hot identifier encodes the opponent archetype directly in the observation, and (B) implicit opponent modeling, in which the agent infers opponent behavior from a rolling window of behavioral statistics. Both agents are trained using MaskablePPO [1] within a custom PettingZoo [2] environment. We further apply Optuna-based hyperparameter optimization [3] and reward shaping to improve convergence. The two models are evaluated against three hand-crafted opponent archetypes—Calling Station, Maniac, and Rock—over 10,000 hands each. We analyze “The Tell”: the behavioral signature that Model B learns to exploit based solely on an opponent’s draw-count pattern, without access to any explicit opponent identity. Our results show that Model B (Implicit) achieves +166.50 BB/100 on average across all archetypes, compared to +180.65 BB/100 for Model A (Explicit), a gap that is statistically significant ($p = 0.029$) yet practically modest, and that Model B outperforms an oracle-labeled explicit agent in a non-stationary setting (+160.35 vs. +156.28 BB/100).

Index Terms—reinforcement learning, opponent modeling, poker AI, MaskablePPO, imperfect-information games, reward shaping, behavioral inference, hyperparameter optimization

I. INTRODUCTION AND MOTIVATION

Poker has long served as a proving ground for AI research on decision-making under uncertainty. Unlike chess or Go, poker involves hidden information—players observe only their own cards and the shared betting history—creating a setting where an optimal strategy must reason about the probability distributions over unobserved states and simultaneously adapt to the behavioral tendencies of the opponent. This combination of imperfect information, sequential decision-making, and strategic adaptation makes poker a rich and challenging domain for reinforcement learning (RL).

Landmark systems such as Libratus [4], DeepStack [5], and Pluribus [6] have achieved superhuman performance in Texas Hold’em by combining Counterfactual Regret Minimization (CFR) [7] with sophisticated endgame solving. While these

systems represent the state of the art in game-theoretically optimal play, they are computationally expensive, require large-scale game-tree traversal, and are primarily designed to resist exploitation rather than to actively exploit opponent weaknesses.

In contrast, this work investigates *exploitative* RL: training a deep policy network to identify and profit from specific, predictable patterns in opponent behavior. We focus on 5-Card Draw Poker, a historically significant variant that has received less attention in the modern deep learning era, and we formulate a concrete research question: *Can a reinforcement learning agent adaptively exploit behaviorally distinct opponents through implicit modeling alone, without access to explicit opponent identity information?*

To answer this question, we design and implement CardShark-RL, a complete RL training and evaluation pipeline. The system includes a custom PettingZoo AEC environment for limit heads-up 5-Card Draw Poker with three hand-coded opponent archetypes. Model A (Explicit) is a MaskablePPO agent that receives a one-hot encoding of the opponent archetype in its observation vector. Model B (Implicit) receives only a rolling window of 10 behavioral statistics computed from past hands, with no direct opponent identity signal. We further apply Optuna-based hyperparameter optimization with a Rock-weighted objective, reward shaping to encourage aggressive exploitation, and a behavioral matrix analysis—“The Tell”—that reveals how Model B adapts its post-draw betting strategy based solely on opponent draw counts.

The remainder of this paper is organized as follows. Section II reviews related work. Section III describes the game environment and opponent archetypes. Section IV details the proposed methodology. Sections V–VII present results, discussion, and conclusions.

II. BACKGROUND AND RELATED WORK

A. Game-Theoretic Poker AI

The challenge of building superhuman poker AI has driven decades of research. Early milestones include Polaris [7],

which applied CFR variants to limit Texas Hold'em. The breakthrough work of Libratus [4] achieved superhuman performance in heads-up no-limit Texas Hold'em in 2017 using an endgame solver and safe subgame-refinement techniques [8]. DeepStack [5] concurrently demonstrated that deep neural networks could replace the look-up tables used in endgame solving, enabling real-time play without pre-computed strategies. Pluribus [6] subsequently extended this paradigm to six-player no-limit Hold'em, the first superhuman AI to master multiplayer poker. A key distinction of these systems is their focus on *equilibrium strategies*—policies robust to any opponent—rather than *exploitative strategies* that target specific opponent weaknesses.

B. Deep Reinforcement Learning in Games

Deep reinforcement learning has achieved remarkable results in a wide range of games. Mnih et al. [9] demonstrated that DQN could achieve human-level performance on Atari games by combining Q-learning with deep convolutional networks, experience replay, and target networks. Silver et al. [10] applied deep RL and Monte Carlo tree search to master the game of Go, a domain previously thought intractable for AI. The Proximal Policy Optimization (PPO) algorithm [11], introduced by Schulman et al., provides a stable and sample-efficient policy gradient method well-suited to discrete action environments such as card games. PPO's clipped surrogate objective prevents destructively large policy updates while remaining simpler to implement than its predecessor, TRPO. In our work, we use MaskablePPO, an extension of PPO from the sb3-contrib library [1], which incorporates action masking to handle the structured invalid-action spaces that arise in poker (betting during draw phases and vice versa).

C. Action Masking in Reinforcement Learning

Huang and Ontañón [12] provide a theoretical and empirical analysis of invalid action masking in policy gradient algorithms. They demonstrate that masking is critical when the ratio of invalid to valid actions is large, as penalty-based approaches fail to learn in such settings. In 5-Card Draw Poker, the agent faces a discrete action space of 35 actions (Fold, Call, Raise, and 32 draw bitmask combinations), of which only a small subset is legal at any given game state. Action masking is therefore essential for efficient learning and training stability.

D. Opponent Modeling and Behavioral Inference

Opponent modeling in multi-agent settings has been studied extensively, with approaches ranging from Bayesian inference over opponent type distributions to neural network-based latent representations. Explicit modeling approaches build a dedicated representation of the opponent (e.g., type classification or hand-range estimation), while implicit approaches allow the agent's policy to adapt to opponent behavior through reward-maximizing experience without an explicit opponent model. The rolling-statistics approach employed by Model B, tracking metrics such as VPIP (Voluntarily Put In Pot), Pre-draw Raise frequency, and Aggression Factor, aligns with the behavioral

profiling tradition in poker theory and has parallels with the latent opponent-embedding approaches studied in recent deep multi-agent RL literature.

E. Hyperparameter Optimization

Hyperparameter optimization (HPO) is critical for RL systems, where training outcomes are sensitive to learning rates, architecture choices, and exploration-exploitation trade-offs. Akiba et al. [3] introduced Optuna, a next-generation HPO framework using tree-structured Parzen estimators (TPE) for efficient Bayesian search. We employ Optuna's MedianPruner with warm-up steps to prune underperforming trials early, enabling efficient search over a 15-dimensional hyperparameter space.

F. Reward Shaping

Ng et al. [13] established the theoretical foundation for potential-based reward shaping, proving that shaped rewards preserve the set of optimal policies under suitable conditions. In poker, raw chip profit is a natural but sparse reward signal. We introduce two auxiliary shaping terms—a penalty for folding when checking was available, and a bonus for successful steal attempts—to guide the agent toward more aggressive exploitation strategies.

III. ENVIRONMENT DESIGN

A. 5-Card Draw Poker Rules

CardShark-RL implements limit heads-up 5-Card Draw Poker, a two-player variant in which each player is dealt five private cards and proceeds through four phases. First, both players post a forced bet (ante) of 1 chip. Second, a pre-draw betting round takes place using a small bet of 2 chips with up to 4 raises. Third, each player selects 0–5 cards to discard and receives replacements from the deck. Fourth, a post-draw betting round uses a big bet of 4 chips with up to 4 raises, after which a showdown determines the winner by best 5-card hand. A player may win before showdown by causing the opponent to fold.

B. PettingZoo AEC Environment

The game is implemented as a PettingZoo Agent-Environment Cycle (AEC) environment [2], a standardized API for turn-based multi-agent systems. The AEC model alternates agent turns in a well-defined cycle, making it natural for sequential card games. The environment is wrapped by a Gymnasium-compatible [14] single-agent wrapper (DrawPokerGymEnv) that auto-steps the opponent, presenting the RL agent (`player_0`) with a standard (`obs`, `reward`, `done`, `info`) interface.

C. Action Space

The action space is `Discrete(35)`. Action 0 is Fold, Action 1 is Call/Check, and Action 2 is Raise/Bet. Actions 3–34 are draw actions, where the index encodes a 5-bit bitmask over the five card positions (bit $i = 1$ discards card i). Action 3 corresponds to bitmask 0 (stand pat; discard nothing) and

Action 34 corresponds to bitmask 31 (discard all five cards). An action mask vector of 35 binary flags is maintained at each step, enabling MaskablePPO [12] to restrict the policy to legal actions. During betting phases, only Actions 0–2 are legal; during the draw phase, only Actions 3–34 are legal.

D. Observation Space

The observation vector is a 26-dimensional (for Model A) or 33-dimensional (for Model B) float32 array normalized to $[0, 1]$ (or $[-1, 1]$ for opponent-draw features). The core 23 features shared by both models are organized into six groups. Hand encoding (10 features) represents each of the five cards as a normalized rank and normalized suit, both in $[0, 1]$. Hand strength (2 features) gives a normalized hand category (0=High Card to 8=Straight Flush) and a normalized evaluation score. Pot features (3 features) include normalized pot size, normalized bet-to-call, and pot odds ratio. Phase encoding (3 features) is a one-hot vector over {pre-draw, draw, post-draw}. Opponent draw count (1 feature) is the normalized number of cards the opponent drew, or -1 if the draw phase has not yet occurred. Strategic features (4 features) cover a positional indicator, own raise count (normalized), opponent aggression flag, and opponent pre-draw raise flag.

Model A appends a 3-dimensional one-hot vector encoding the opponent archetype (Calling Station, Maniac, or Rock).

Model B appends a 10-dimensional rolling statistics vector computed over the last W hands, comprising: fold frequency pre-draw, fold frequency post-draw, raise frequency post-draw, average cards drawn (normalized), raise-then-stand-pat correlation, fold-after-drawing-3 frequency, raise-after-standing-pat frequency, VPIP (Voluntarily Put In Pot), PFR (Pre-draw Raise frequency), and Aggression Factor (raises / calls, normalized).

E. Hand Evaluator

Cards are encoded as integers 0–51 where $\text{rank} = c \bmod 13$ (0=2, ..., 12=A) and $\text{suit} = \lfloor c/13 \rfloor$ (0=Clubs, ..., 3=Spades). Hand evaluation returns a scalar score = category $\times 10^6$ + tiebreaker, where the tiebreaker uses base-14 encoding of rank groups, ensuring score ordering within and across all nine hand categories (High Card through Straight Flush).

F. Opponent Archetypes

Three hand-crafted deterministic/stochastic opponents model behaviorally distinct playing styles.

CallingStation (Archetype 0) is a passive opponent that never bluffs and never raises. It always calls and draws optimally by keeping its best group (pair, trips, etc.) and discarding kickers.

Maniac (Archetype 1) is an aggressive opponent that raises 70% of the time when raising is legal, and chases flush and straight draws 60% of the time during the draw phase by drawing one card. It rarely folds (5% probability), creating a loose, unpredictable style.

Rock (Archetype 2) is a tight-passive opponent that folds pre-draw on all unpaired hands if there is a bet to call, calls only with a pair below Jacks, raises with Jacks-or-better, and

stands pat on two-pair or better hands. This opponent is the hardest to exploit, as it only invests chips with strong holdings.

IV. METHODOLOGY

A. Training Algorithm: MaskablePPO

Both models are trained using MaskablePPO from sb3-contrib [1], a variant of Proximal Policy Optimization [11] that incorporates action masking directly into the policy gradient update. The policy is an MLP (Multi-Layer Perceptron) actor-critic network. The clipped surrogate objective of PPO is:

$$\mathcal{L}^{\text{CLIP}}(\theta) = \hat{\mathbb{E}}_t \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right) \right] \quad (1)$$

where $r_t(\theta) = \pi_\theta(a_t|s_t)/\pi_{\theta_{\text{old}}}(a_t|s_t)$ is the probability ratio and $\hat{A}_t$ is the Generalized Advantage Estimate (GAE) [15]. The GAE with parameter λ interpolates between one-step TD error ($\lambda = 0$) and Monte Carlo returns ($\lambda = 1$):

$$\hat{A}_t = \sum_{l=0}^{\infty} (\gamma\lambda)^l \delta_{t+l}, \quad \delta_t = r_t + \gamma V(s_{t+1}) - V(s_t) \quad (2)$$

Action masking is applied at inference time by setting the log-probabilities of illegal actions to $-\infty$ before the softmax, ensuring the agent never samples illegal actions. Huang and Onta  n [12] demonstrate that this hard masking is both theoretically sound and empirically superior to penalty-based alternatives.

B. Network Architecture

Both models use an MLP policy with fully connected layers of equal width. The policy and value networks share no parameters; each is a separate MLP head on top of the shared feature extractor. For Model B (implicit), the optimized architecture uses four hidden layers of 256 neurons each (depth chosen by HPO). For Model A (explicit), a three-layer architecture of 256 neurons is used as the default.

C. Reward Function

The base reward at the end of each hand is the net chip profit: $r = \text{pot_won} - \text{chips_invested}$ for the winner, and $r = -\text{chips_invested}$ for the loser. Ties yield $r = 0$.

Two shaping terms [13] are added. A fold penalty ($-\alpha$) is applied when the agent folds during a phase in which checking would have been free (bet-to-call = 0), penalizing unnecessarily surrendered equity. A steal bonus ($+\beta$) is applied when the agent raises and the opponent subsequently folds (a successful steal), rewarding aggressive pot-theft. The shaping weights α and β are treated as hyperparameters and tuned by Optuna. For Model B, the optimized values are $\alpha = 0.296$ and $\beta = 0.448$.

D. Opponent Scheduling

For Model A (explicit), random opponent scheduling is used: a new archetype is chosen uniformly at random before each episode. Since the agent has direct access to the opponent’s identity through the one-hot feature, it can immediately condition its policy on the current archetype.

For Model B (implicit), a *hybrid schedule* is employed. The first H episodes use *block scheduling*: opponents are cycled through in blocks of B consecutive hands, allowing the rolling statistics to build up a stable behavioral signal before the agent can exploit it. After H episodes, random scheduling is resumed to encourage robustness. The switch point H and block size B are chosen by HPO.

E. Hyperparameter Optimization with Optuna

We employ Optuna [3] with Tree-structured Parzen Estimator (TPE) sampling and a MedianPruner to search over a 15-dimensional hyperparameter space. The best configurations found for both models are summarized in Table I. Each trial trains the model for 300,000 timesteps and is evaluated using a Rock-weighted objective:

$$\text{score} = 0.5 \cdot \text{BB}_{100}^{\text{Rock}} + 0.25 \cdot \text{BB}_{100}^{\text{Station}} + 0.25 \cdot \text{BB}_{100}^{\text{Maniac}} \quad (3)$$

The Rock archetype receives extra weight (0.5) because it is the hardest opponent to exploit and determines whether the agent has learned genuine strategic depth or is merely farming easy wins against passive opponents.

TABLE I
BEST HYPERPARAMETERS FOR MODEL A (EXPLICIT) AND MODEL B (IMPLICIT)

Hyperparameter	Model A (Explicit)	Model B (Implicit)
learning_rate	1.63×10^{-4}	3.50×10^{-4}
n_steps	4096	2048
batch_size	32	64
n_epochs	22	11
gamma	0.9658	0.9894
ent_coef	0.00103	0.00457
clip_range	0.242	0.194
gae_lambda	0.874	0.922
vf_coef	0.475	0.430
max_grad_norm	0.484	0.464
fold_penalty	0.379	0.328
steal_bonus	0.492	0.212
lr_schedule	constant	linear
net_arch	[128, 128]	[256, 256, 256, 256]
rolling_window W	N/A	50
block_size B	N/A	400
hybrid_switch H	N/A	972,000 steps

F. Training Protocol

Both models are trained for 1,000,000 timesteps using $n_{\text{envs}} = 4$ parallel environments (DummyVecEnv) with distinct random seeds. A per-opponent evaluation callback runs every 50,000 timesteps, reporting intermediate BB/100 values for each archetype to monitor learning dynamics. Training is performed on a single machine; the device is selected automatically (CPU or CUDA).

G. Evaluation Metric: BB/100

Performance is reported in *big blinds per 100 hands* (BB/100), a standard poker metric:

$$\text{BB/100} = \frac{\text{total profit}}{\text{number of hands}} \times 100 \quad (4)$$

where the big blind equals 4 chips (the post-draw bet size) in our game. A positive BB/100 indicates a profitable strategy; a negative value indicates a losing strategy. Each evaluation run uses 10,000 hands against a fixed opponent archetype with a warm-up phase of 50 random hands (for Model B only) to populate the rolling statistics buffer before evaluation begins.

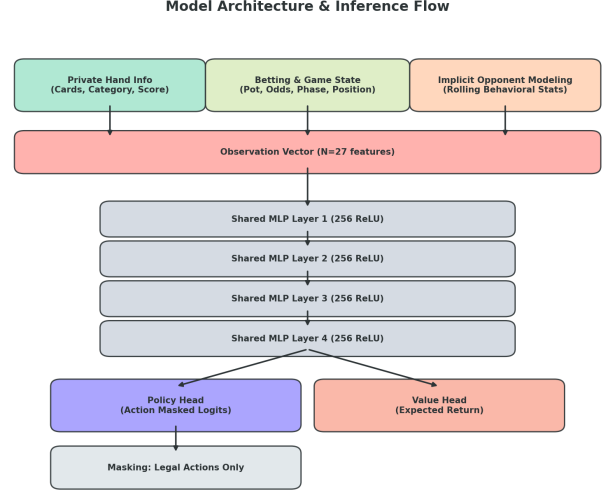


Fig. 1. Model B (Implicit) architecture and inference flow. The observation vector concatenates private hand information, game-state features, and the 10-dimensional rolling behavioral statistics. Four shared MLP layers (256 ReLU units each) feed separate policy and value heads; action masking is applied before sampling.

V. EXPERIMENTAL RESULTS

A. Cross-Evaluation Performance

Both models are evaluated against each of the three opponent archetypes over 10,000 hands, repeated across three independent evaluation seeds (0, 100, 200). Mean BB/100 and standard deviations are reported in Table II. A random-action baseline is included for reference.

TABLE II
EVALUATION RESULTS: BB/100 BY MODEL AND OPPONENT (MEAN $\pm$ STD OVER 3 SEEDS)

Opponent	Model A (Expl.)	Model B (Impl.)	Random
CallingStation	$+86.5 \pm 4.3$	$+79.9 \pm 5.0$	-143.8 ± 2.7
Maniac	$+408.0 \pm 15.7$	$+390.8 \pm 11.5$	-222.1 ± 0.1
Rock	$+47.5 \pm 2.8$	$+28.9 \pm 6.5$	-151.3 ± 2.0
Average	+180.65	+166.50	-172.44

Both trained agents substantially outperform the random baseline across all archetypes. Model A (Explicit) posts the higher average (+180.65 BB/100 vs. +166.50 BB/100 for

Model B), which is expected given that it receives the opponent identity directly in its observation. The gap is most pronounced against the Rock archetype (+47.5 vs. +28.9), a hard-to-exploit opponent that only invests chips with strong holdings. Against the Maniac, both models achieve very high profitability, as the Maniac’s loose aggression and frequent draws create many exploitable spots.

Fig. 2 shows the cross-evaluation profitability as a grouped bar chart, and Fig. 3 shows cumulative profit trajectories over 30,000 hands (three seeds concatenated). The linear growth of the profit curves for both trained agents confirms that their exploitative advantage is stable and does not degrade over longer horizons.

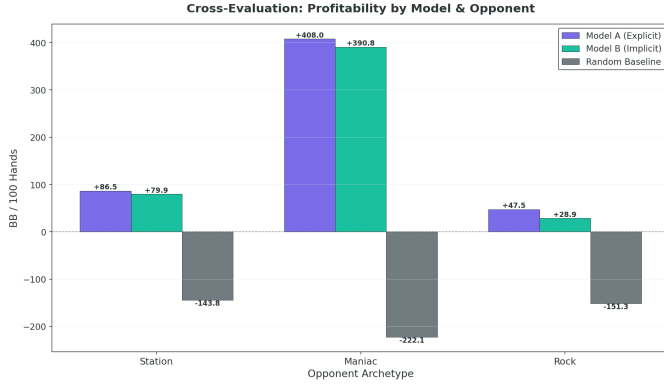


Fig. 2. Cross-evaluation profitability (BB/100) for Model A (Explicit), Model B (Implicit), and the random baseline across the three opponent archetypes. Error bars are not shown in the bar chart but are provided in Table II.

B. Behavioral Analysis: “The Tell”

A central contribution of this work is the behavioral matrix analysis of Model B’s post-draw action distribution, conditioned on the observed opponent draw count. Fig. 4 displays the frequency of Fold, Call, and Raise actions for three representative draw-count values (0, 1, and 3) across each opponent archetype.

Several patterns emerge from the matrix. Against the CallingStation, Model B maintains consistently high call and raise frequencies regardless of draw count (0% fold rate when the station stands pat or draws one), reflecting the station’s predictably passive post-draw behavior. Against the Maniac, the model increases its raise frequency when the Maniac draws one card (52%), as a single-card draw from a loose opponent is a common flush or straight chase that frequently misses. The most striking pattern appears against the Rock: when the Rock stands pat (Draw 0), Model B responds with 50% call and 43% raise, treating a Rock stand-pat as a strong hand signal. When the Rock draws one card, Model B’s call and raise frequencies drop to 0%, indicating learned recognition that this is an unusual Rock action associated with high hand strength. These patterns demonstrate that Model B has learned opponent-specific hand-reading heuristics from draw-count telemetry alone, without any explicit archetype identifier in its input.

C. Non-Stationary Adaptation

To evaluate Model B’s ability to adapt when the opponent changes mid-session, we run a 1,500-hand tournament in which the opponent switches from CallingStation (hands 1–500) to Maniac (hands 501–1,000) to Rock (hands 1,001–1,500) without resetting Model B’s rolling statistics buffer. Two variants of Model A are also evaluated: an *Oracle* variant given the correct archetype label at each switch, and a *Blind* variant given a fixed incorrect label throughout.

Results of the tournament are summarized in Table III. Model B achieves +160.35 BB/100, outperforming Model A-Oracle (+156.28 BB/100) and substantially outperforming Model A-Blind (+50.01 BB/100). Fig. 5 shows that the rolling statistics (VPIP, PFR, Aggression Factor, Avg. Cards Drawn) adapt to the new opponent within approximately 50–100 hands after each switch. Fig. 6 shows the corresponding win-rate curve, with brief transients at each switch point followed by recovery.

TABLE III
NON-STATIONARY ADAPTATION TOURNAMENT RESULTS (1,500 HANDS)

Agent	BB/100
Model B (Implicit)	+160.35
Model A (Explicit – Oracle)	+156.28
Model A (Explicit – Blind)	+50.01

D. Statistical Validation

Statistical significance of Model B’s performance is assessed over 270,000 pooled hands (three opponents $\times$ three seeds $\times$ 10,000 hands each). A paired t -test comparing per-hand profit between Model B and Model A yields $t = -2.183$, $p = 0.029$, indicating that the difference in means is statistically significant at the $\alpha = 0.05$ level. Model B vs. the random baseline is highly significant ($t = +106.85$, $p \approx 0$). A 95% confidence interval for Model B’s average per-hand profit gives +1.71 chips [CI: 1.65, 1.76], corresponding to +170.87 BB/100 [CI: 165.31, 176.44].

VI. DISCUSSION

A. Implicit vs. Explicit Modeling

The performance gap between Model A and Model B—roughly 14 BB/100 on average—is consistent with the information asymmetry: Model A receives the opponent archetype as a direct input, while Model B must infer it from behavioral evidence accumulated over recent hands. The gap is largest against the Rock (+18.6 BB/100 in favor of Model A), whose tight-passive style generates sparse exploitable signals. Against the Maniac, the gap narrows considerably (+17.2 BB/100), suggesting that the Maniac’s high VPIP and frequent draws are sufficient to produce a strong implicit signal within the rolling window of $W = 50$ hands.

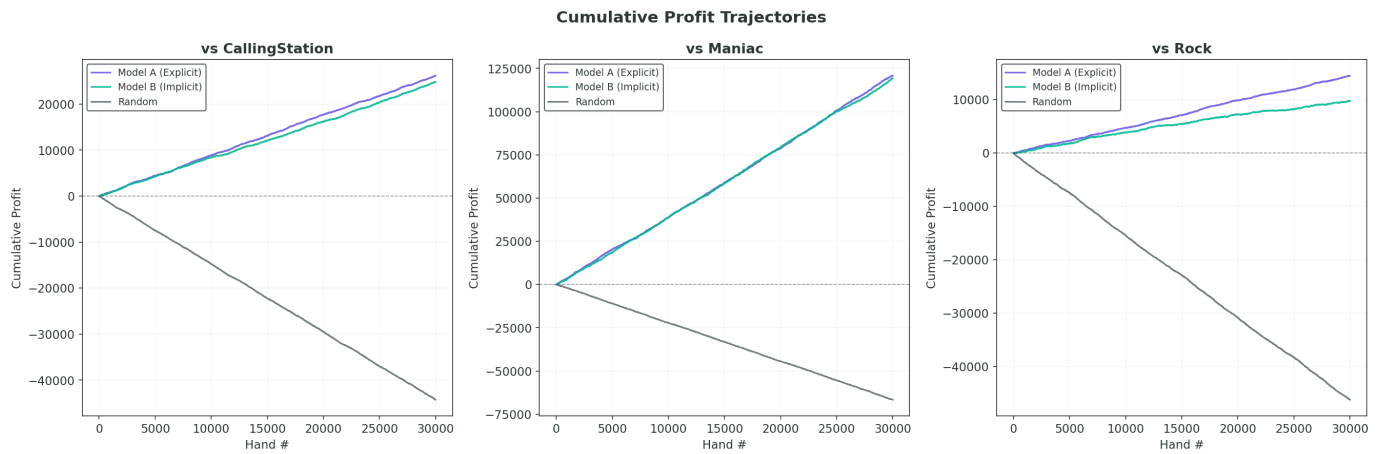


Fig. 3. Cumulative profit trajectories over 30,000 hands (three 10,000-hand evaluation seeds concatenated) per opponent archetype. Both trained models exhibit consistent positive linear growth, while the random baseline degrades uniformly.

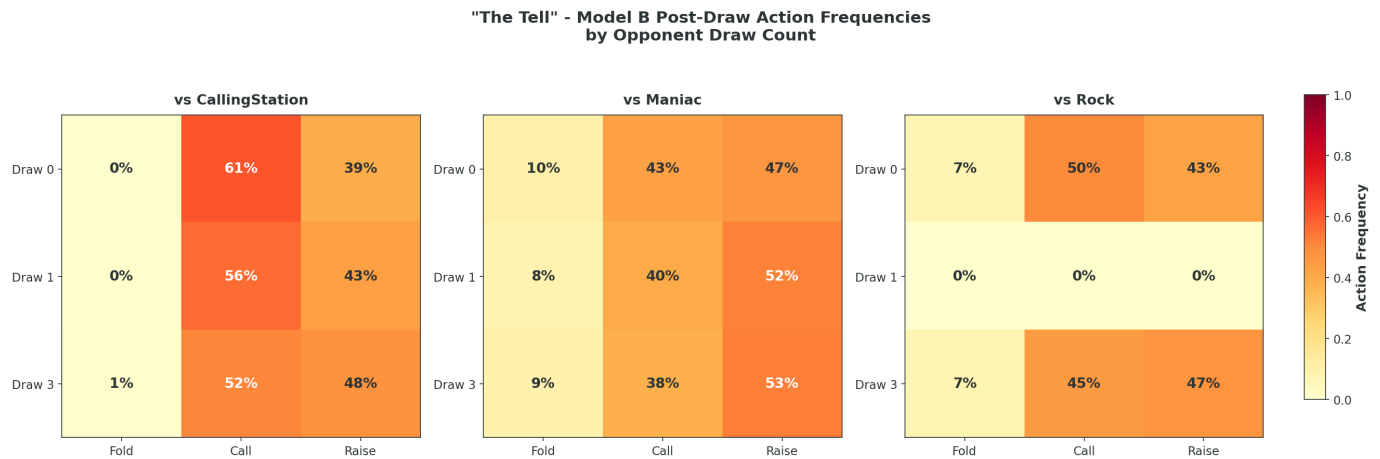


Fig. 4. "The Tell": Model B post-draw action frequencies conditioned on opponent draw count (rows) for each opponent archetype (panels). Model B adapts its post-draw aggression based on opponent draw count without access to explicit opponent identity.

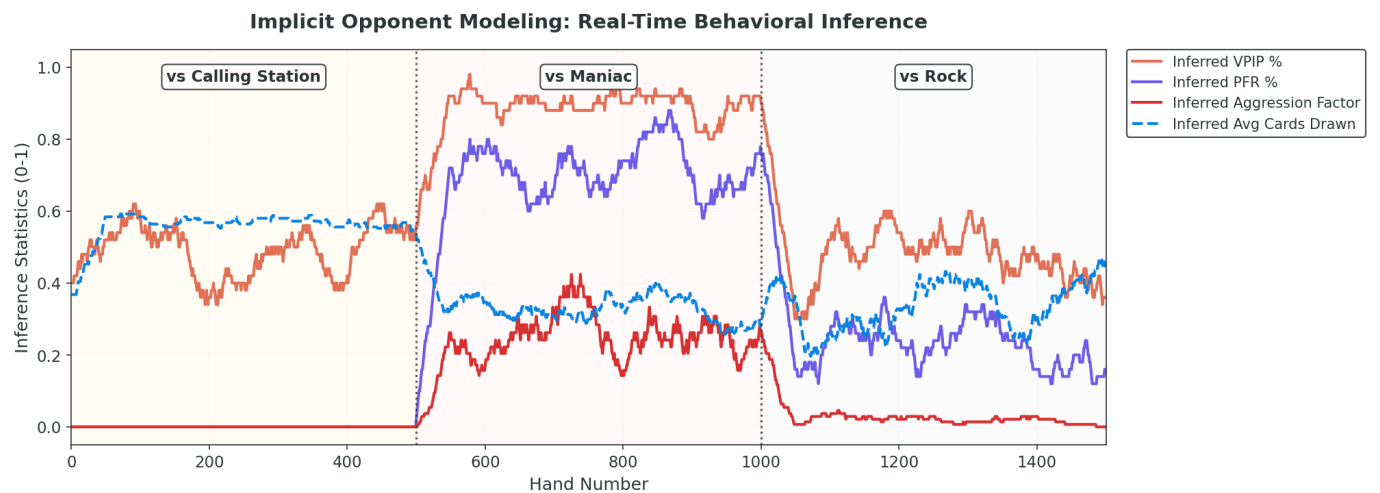


Fig. 5. Model B's rolling behavioral statistics (VIP, PFR, Aggression Factor, Avg. Cards Drawn) across the 1,500-hand non-stationary tournament. Vertical dashed lines mark opponent switches. The statistics adapt within ~50-100 hands after each switch.

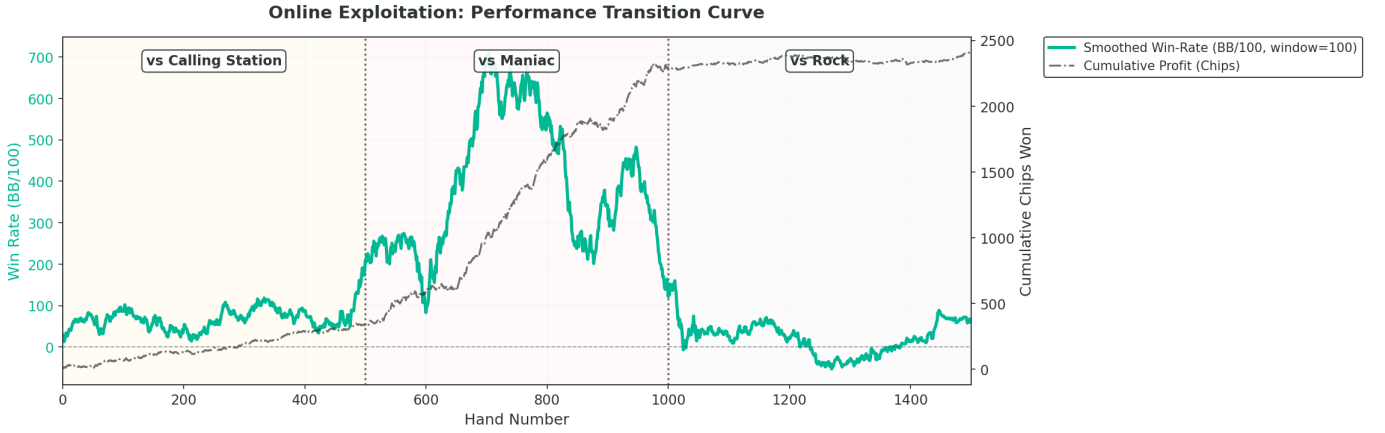


Fig. 6. Online exploitation performance of Model B in the non-stationary tournament: smoothed win rate (BB/100, window=100) and cumulative profit. Win rate peaks during the Maniac segment and recovers after each opponent switch with a brief transient.

B. The Non-Stationary Advantage

Somewhat counterintuitively, Model B slightly outperforms Model A-Oracle in the non-stationary tournament (+160.35 vs. +156.28 BB/100). One plausible explanation is that Model B’s rolling statistics provide a more nuanced, *continuous* representation of opponent behavior than a single one-hot identity label. When the opponent plays a mix of strategies at the boundary of archetype definitions, the rolling statistics can capture subtle intermediate states that the discrete label cannot. This interpretation aligns with the behavioral matrix in Fig. 4, which shows that Model B conditions on draw-count in an archetype-specific manner.

C. Limitations

Several limitations bound the scope of our conclusions. First, all opponents are hand-coded deterministic or fixed-stochastic archetypes; the agent has not been tested against adaptive or self-play opponents. Second, the training environment uses limit betting with a fixed ante, which removes no-limit strategic considerations such as stack-size-aware betting. Third, the rolling statistics window size ($W = 50$) requires several dozen hands before the behavioral signal stabilizes, which imposes a cold-start cost when the opponent switches. Finally, the evaluation is restricted to heads-up play; generalization to multi-player settings would require significant architectural changes.

VII. CONCLUSION

We presented CardShark-RL, a reinforcement learning system for heads-up 5-Card Draw Poker that investigates two approaches to opponent adaptation. Model A uses explicit opponent identity to condition its policy, while Model B infers opponent behavior from a rolling window of behavioral statistics. Both models substantially outperform a random baseline across all tested archetypes. Model B achieves an average of +166.50 BB/100, compared to +180.65 BB/100 for Model A, with the gap being statistically significant ($p = 0.029$) but

practically modest given that Model B has no access to opponent identity.

The behavioral matrix analysis reveals that Model B has learned opponent-specific hand-reading heuristics from draw-count telemetry alone. In the non-stationary tournament, Model B marginally outperforms Model A-Oracle (+160.35 vs. +156.28 BB/100), indicating that implicit behavioral inference can be more robust than explicit labeling when the environment is non-stationary.

Future work could extend this framework to no-limit betting, self-play training, and multi-player settings. Incorporating uncertainty estimates over the opponent’s hand range—combining the implicit behavioral statistics with Bayesian hand-range inference—could further narrow the gap with explicit modeling, particularly against tight-passive opponents like the Rock.

REFERENCES

- [1] A. Raffin, A. Hill, A. Gleave, A. Kanervisto, M. Ernestus, and N. Dormann, “Stable-baselines3: Reliable reinforcement learning implementations,” *Journal of Machine Learning Research*, vol. 22, no. 268, pp. 1–8, 2021. [Online]. Available: <http://jmlr.org/papers/v22/20-1364.html>
- [2] J. K. Terry, B. Black, N. Grammel, M. Jayakumar, A. Hari, R. Sullivan, L. S. Santos, C. Dieffendahl, C. Horsch, R. Perez-Vicente, N. L. Williams, Y. Lokesh, and P. Ravi, “Pettingzoo: Gym for multi-agent reinforcement learning,” *arXiv preprint arXiv:2009.14471*, 2021. [Online]. Available: <https://arxiv.org/abs/2009.14471>
- [3] T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama, “Optuna: A next-generation hyperparameter optimization framework,” in *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, 2019, pp. 2623–2631.
- [4] N. Brown and T. Sandholm, “Superhuman AI for heads-up no-limit poker: Libratus beats top professionals,” *Science*, vol. 359, no. 6374, pp. 418–424, 2018.
- [5] M. Moravčík, M. Schmid, N. Burch, V. Lisý, D. Morrill, N. Bard, T. Davis, K. Waugh, M. Johanson, and M. Bowling, “DeepStack: Expert-level artificial intelligence in heads-up no-limit poker,” *Science*, vol. 356, no. 6337, pp. 508–513, 2017.
- [6] N. Brown and T. Sandholm, “Superhuman AI for multiplayer poker,” *Science*, vol. 365, no. 6456, pp. 885–890, 2019.
- [7] M. Zinkevich, M. Johanson, M. Bowling, and C. Piccione, “Regret minimization in games with incomplete information,” in *Advances in Neural Information Processing Systems*, vol. 20, 2007.

- [8] S. Ganzfried and T. Sandholm, "Endgame solving in large imperfect-information games," in *Proceedings of the 2015 International Conference on Autonomous Agents and Multiagent Systems*, 2015, pp. 37–45.
- [9] V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. Riedmiller, A. K. Fidjeland, G. Ostrovski, S. Petersen, C. Beattie, A. Sadik, I. Antonoglou, H. King, D. Kumaran, D. Wierstra, S. Legg, and D. Hassabis, "Human-level control through deep reinforcement learning," *Nature*, vol. 518, no. 7540, pp. 529–533, 2015.
- [10] D. Silver, A. Huang, C. J. Maddison, A. Guez, L. Sifre, G. van den Driessche, J. Schrittwieser, I. Antonoglou, V. Panneershelvam, M. Lanctot, S. Dieleman, D. Grewe, J. Nham, N. Kalchbrenner, I. Sutskever, T. Lillicrap, M. Leach, K. Kavukcuoglu, T. Graepel, and D. Hassabis, "Mastering the game of Go with deep neural networks and tree search," *Nature*, vol. 529, no. 7587, pp. 484–489, 2016.
- [11] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," in *arXiv preprint arXiv:1707.06347*, 2017. [Online]. Available: <https://arxiv.org/abs/1707.06347>
- [12] S. Huang and S. Ontañón, "A closer look at invalid action masking in policy gradient algorithms," in *Proceedings of the 35th International FLAIRS Conference*, 2022.
- [13] A. Y. Ng, D. Harada, and S. Russell, "Policy invariance under reward transformations: Theory and application to reward shaping," in *Proceedings of the Sixteenth International Conference on Machine Learning*, 1999, pp. 278–287.
- [14] G. Brockman, V. Cheung, L. Pettersson, J. Schneider, J. Schulman, J. Tang, and W. Zaremba, "Openai gym," 2016.
- [15] J. Schulman, P. Moritz, S. Levine, M. Jordan, and P. Abbeel, "High-dimensional continuous control using generalized advantage estimation," in *International Conference on Learning Representations*, 2016. [Online]. Available: <https://arxiv.org/abs/1506.02438>